Aula 4 — EDA Profissional (Versão Avançada) - Para Aula

Dataset: Airbnb NYC

Objetivos Desta Aula

- **Explorar um dataset real de mercado:** Entender a estrutura e o conteúdo de dados reais.
- **Detectar e tratar valores faltantes e outliers:** Aprender técnicas para limpar e preparar os dados.
- **Realizar análise por categoria e por localização:** Investigar padrões e tendências em diferentes segmentos e regiões.
- **Construir gráficos profissionais e informativos:** Utilizar bibliotecas avançadas para visualizações de alta qualidade.
- **Introduzir correlação entre variáveis:** Entender as relações entre diferentes atributos do dataset.
- **Engenharia de Features:** Criar novas variáveis para enriquecer a análise.
- **Análise Geoespacial:** Visualizar dados em um contexto geográfico.
- **Interpretar e comunicar insights:** Extrair conclusões significativas dos dados.

1) Importação das bibliotecas Essenciais

Nesta seção, importamos todas as bibliotecas Python necessárias para a Análise Exploratória de Dados (EDA).

- `pandas` : Para manipulação e análise de dados em formato de DataFrame.
- `matplotlib.pyplot` : Para a criação de gráficos estáticos.
- `seaborn` : Baseado no matplotlib, oferece uma interface de alto nível para criar gráficos estatísticos atraentes e informativos.
- `numpy` : Para operações numéricas e matemáticas, especialmente útil para lidar com arrays e cálculos estatísticos.
- `folium` : Para criar mapas interativos e visualizar dados geoespaciais.

In [96]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import folium
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['axes.titlesize'] = 14
```

2) Carregar o Dataset

Carregamos o dataset do Airbnb NYC a partir de um arquivo CSV. Este é o primeiro passo para qualquer análise de dados, onde os dados brutos são lidos e transformados em um DataFrame do pandas para fácil manipulação.

In [97]:
```
df = pd.read_csv('AB_NYC_2019.csv')
display(df.head().T)
```

	0	1	2	3	
id	2539	2595	3647	3831	50
name	Clean & quiet apt home by the park	Skylit Midtown Castle	THE VILLAGE OF HARLEM....NEW YORK !	Cozy Entire Floor of Brownstone	Entire A Spacio Studio/L by cent p
host_id	2787	2845	4632	4869	71
host_name	John	Jennifer	Elisabeth	LisaRoxanne	La
neighbourhood_group	Brooklyn	Manhattan	Manhattan	Brooklyn	Manhat
neighbourhood	Kensington	Midtown	Harlem	Clinton Hill	E Harl
latitude	40.64749	40.75362	40.80902	40.68514	40.798
longitude	-73.97237	-73.98377	-73.9419	-73.95976	-73.943
room_type	Private room	Entire home/apt	Private room	Entire home/apt	Ent home/
price	149	225	150	89	
minimum_nights	1	1	3	1	
number_of_reviews	9	45	0	270	
last_review	2018-10-19	2019-05-21	NaN	2019-07-05	2018-
reviews_per_month	0.21	0.38	NaN	4.64	
calculated_host_listings_count	6	2	1	1	
availability_365	365	355	365	194	

3) Visão Geral e Estrutura dos Dados

Esta seção fornece uma visão geral rápida do dataset, incluindo suas dimensões, tipos de dados e um resumo estatístico. Isso nos ajuda a entender a estrutura dos dados e identificar possíveis problemas, como valores ausentes ou tipos de dados incorretos.

In [98]:
```
# Exiba o número de linhas e colunas do DataFrame (df.shape)
print("Numero de linhas e colunas(linhas,colunas):", df.shape)
print("-" * 40)
# Exiba informações sobre o DataFrame, incluindo tipos de dados e valores não-nu
df.info()
print("-" * 40)
# Exiba estatísticas descritivas para as colunas numéricas (df.describe())
print("Estatisticas descritivas (numericas):")
display(df.describe().T)
print("-" * 40)
# Exiba estatísticas descritivas para todas as colunas (incluindo categóricas) (
print("Estatisticas descritivas (Todas as colunas):")
display(df.describe(include="all").T)
```

```
Numero de linhas e colunas(linhas,colunas): (48895, 16)
----------------------------------------
<class 'pandas.DataFrame'>
RangeIndex: 48895 entries, 0 to 48894
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              48895 non-null  int64
 1   name                            48879 non-null  str
 2   host_id                         48895 non-null  int64
 3   host_name                       48874 non-null  str
 4   neighbourhood_group             48895 non-null  str
 5   neighbourhood                   48895 non-null  str
 6   latitude                        48895 non-null  float64
 7   longitude                       48895 non-null  float64
 8   room_type                       48895 non-null  str
 9   price                           48895 non-null  int64
 10  minimum_nights                  48895 non-null  int64
 11  number_of_reviews               48895 non-null  int64
 12  last_review                     38843 non-null  str
 13  reviews_per_month               38843 non-null  float64
 14  calculated_host_listings_count  48895 non-null  int64
 15  availability_365                48895 non-null  int64
dtypes: float64(3), int64(7), str(6)
memory usage: 6.0 MB
----------------------------------------
Estatisticas descritivas (numericas):
```

	count	mean	std	min	
id	48895.0	1.901714e+07	1.098311e+07	2539.00000	9.471945
host_id	48895.0	6.762001e+07	7.861097e+07	2438.00000	7.822033
latitude	48895.0	4.072895e+01	5.453008e-02	40.49979	4.069010
longitude	48895.0	-7.395217e+01	4.615674e-02	-74.24442	-7.398307
price	48895.0	1.527207e+02	2.401542e+02	0.00000	6.900000
minimum_nights	48895.0	7.029962e+00	2.051055e+01	1.00000	1.000000
number_of_reviews	48895.0	2.327447e+01	4.455058e+01	0.00000	1.000000
reviews_per_month	38843.0	1.373221e+00	1.680442e+00	0.01000	1.90000
calculated_host_listings_count	48895.0	7.143982e+00	3.295252e+01	1.00000	1.000000
availability_365	48895.0	1.127813e+02	1.316223e+02	0.00000	0.000000

```
----------------------------------------
Estatisticas descritivas (Todas as colunas):
```

	count	unique	top	freq	mean	
id	48895.0	NaN	NaN	NaN	19017143.23618	109
name	48879	47905	Hillside Hotel	18	NaN	
host_id	48895.0	NaN	NaN	NaN	67620010.64661	7861
host_name	48874	11452	Michael	417	NaN	
neighbourhood_group	48895	5	Manhattan	21661	NaN	
neighbourhood	48895	221	Williamsburg	3920	NaN	
latitude	48895.0	NaN	NaN	NaN	40.728949	
longitude	48895.0	NaN	NaN	NaN	-73.95217	
room_type	48895	3	Entire home/apt	25409	NaN	
price	48895.0	NaN	NaN	NaN	152.720687	
minimum_nights	48895.0	NaN	NaN	NaN	7.029962	
number_of_reviews	48895.0	NaN	NaN	NaN	23.274466	
last_review	38843	1764	2019-06-23	1413	NaN	
reviews_per_month	38843.0	NaN	NaN	NaN	1.373221	
calculated_host_listings_count	48895.0	NaN	NaN	NaN	7.143982	
availability_365	48895.0	NaN	NaN	NaN	112.781327	

4) Tratamento de Valores Faltantes

Valores faltantes são comuns em datasets reais e podem afetar a qualidade da análise. Nesta seção, identificamos e tratamos esses valores, decidindo entre remover, preencher com um valor específico (média, mediana, moda) ou usar métodos mais avançados.

In [99]:
```
# Calcule e exiba a quantidade de valores faltantes por coluna
print("Valores faltantes por coluna:")
print(df.isnull().sum())
print("-" * 40)
# Preencha os valores faltantes nas colunas `name` e `host_name` com "unknown"
df['name'] = df['name'].fillna("unknown")
df['host_name'] = df['host_name'].fillna("unknown")
# Preencha os valores faltantes na coluna `reviews_per_month` com 0
df['reviews_per_month'] = df['reviews_per_month'].fillna(0)
# Converta a coluna `last_review` para datetime e preencha os valores faltantes
df['last_review'] = pd.to_datetime(df['last_review'])
df['last_review'] = df['last_review'].fillna(pd.to_datetime("2010-01-01"))
# Verifique novamente os valores faltantes após o tratamento
print("Valores apos o tratamento")
print(df.isnull().sum())
print("-" * 40)
```

```
Valores faltantes por coluna:
id                                  0
name                               16
host_id                             0
host_name                          21
neighbourhood_group                 0
neighbourhood                       0
latitude                            0
longitude                           0
room_type                           0
price                               0
minimum_nights                      0
number_of_reviews                   0
last_review                     10052
reviews_per_month               10052
calculated_host_listings_count      0
availability_365                    0
dtype: int64
----------------------------------------
Valores apos o tratamento
id                                0
name                              0
host_id                           0
host_name                         0
neighbourhood_group               0
neighbourhood                     0
latitude                          0
longitude                         0
room_type                         0
price                             0
minimum_nights                    0
number_of_reviews                 0
last_review                       0
reviews_per_month                 0
calculated_host_listings_count    0
availability_365                  0
dtype: int64
----------------------------------------
```

5) Detecção e Tratamento de Outliers

Outliers são pontos de dados que se desviam significativamente de outras observações. Eles podem distorcer análises estatísticas e modelos de machine learning. Nesta seção, identificamos e tratamos outliers em variáveis numéricas chave, como `price` e `minimum_nights`. Usaremos o método do Intervalo Interquartil (IQR) para uma abordagem robusta.

In [100…

```
# Defina uma função `remove_outliers_iqr` para detectar e remover outliers usand
def remove_outliers_iqr(df, coluna):
    Q1= df[coluna].quantile(0.25)
    Q3= df[coluna].quantile(0.75)
    IQR = Q3 - Q1
    limite_inf=Q1 - 1.5*IQR
    limite_sup=Q3 + 1.5*IQR
    df_filtrado= df[(df[coluna] >= limite_inf) & (df[coluna] <= limite_sup)]
    return df_filtrado
# Aplique a função para tratar outliers nas colunas `price`, `minimum_nights` e
```

```
# Exiba as estatísticas descritivas antes e depois do tratamento para cada colun
# Atualize o df principal com os dados limpos
print("Colunas sem tratamento:")
print(df[['price','minimum_nights','number_of_reviews']].describe())
print("-" * 40)
df =remove_outliers_iqr(df,'price')
df =remove_outliers_iqr(df,'minimum_nights')
df =remove_outliers_iqr(df,'number_of_reviews')
print("Apos tratamento:")
print(df[['price','minimum_nights','number_of_reviews']].describe())
```

```
Colunas sem tratamento:
              price  minimum_nights  number_of_reviews
count  48895.000000    48895.000000       48895.000000
mean     152.720687        7.029962          23.274466
std      240.154170       20.510550          44.550582
min        0.000000        1.000000           0.000000
25%       69.000000        1.000000           1.000000
50%      106.000000        3.000000           5.000000
75%      175.000000        5.000000          24.000000
max    10000.000000     1250.000000         629.000000
----------------------------------------
Apos tratamento:
              price  minimum_nights  number_of_reviews
count  35202.000000    35202.000000       35202.000000
mean     119.289643        2.765042          12.362536
std       67.831566        1.917473          16.631476
min        0.000000        1.000000           0.000000
25%       65.000000        1.000000           1.000000
50%      100.000000        2.000000           5.000000
75%      155.000000        3.000000          17.000000
max      334.000000       11.000000          71.000000
```

6) Análise Exploratória de Dados (EDA) Aprofundada

Com os dados limpos, podemos agora aprofundar nossa análise, explorando as distribuições das variáveis, as relações entre elas e identificando padrões que podem gerar insights valiosos.

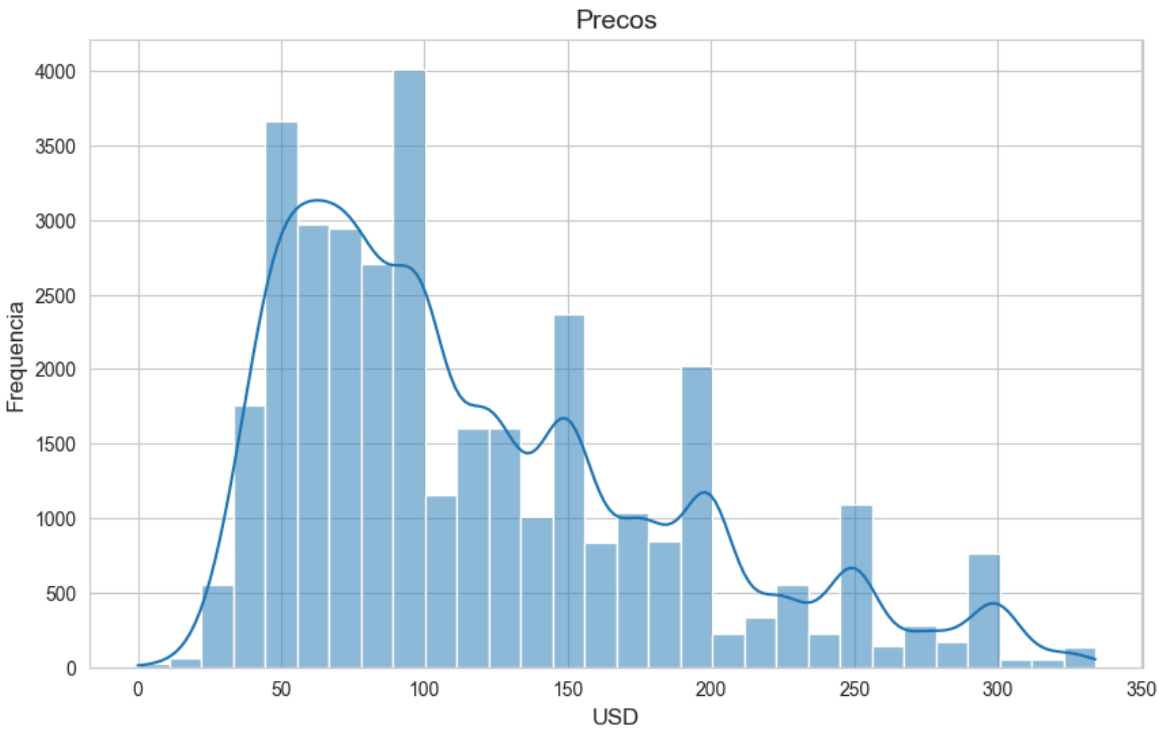
6.1) Distribuição de Preços

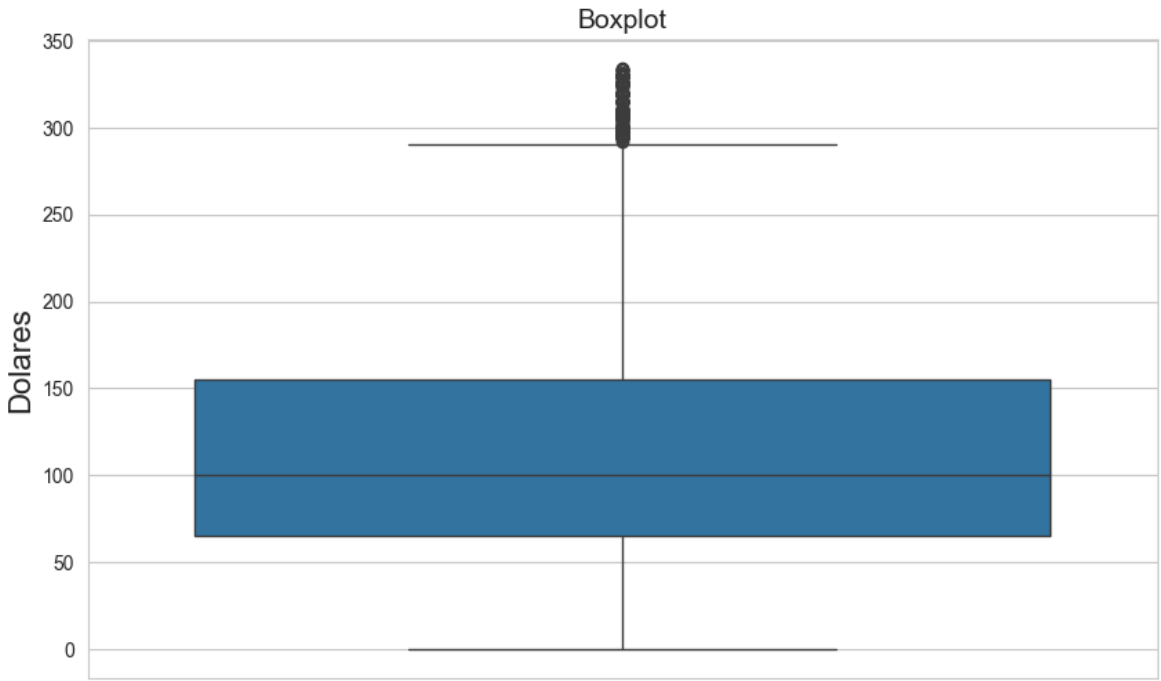
Vamos visualizar a distribuição dos preços dos aluguéis para entender a faixa de valores e identificar a concentração de preços.

In [101…

```
# Crie um histograma da coluna `price` usando `sns.histplot`
# Crie um boxplot da coluna `price` usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(10,6))
sns.histplot(data=df,x='price',kde=True, bins=30)
plt.title("Precos")
plt.xlabel("USD",fontsize=12)
plt.ylabel("Frequencia",fontsize=12)
plt.show()
sns.boxplot(data=df,y='price')
plt.ylabel("Dolares",fontsize=16)
```

```
plt.title("Boxplot")
plt.show()
```





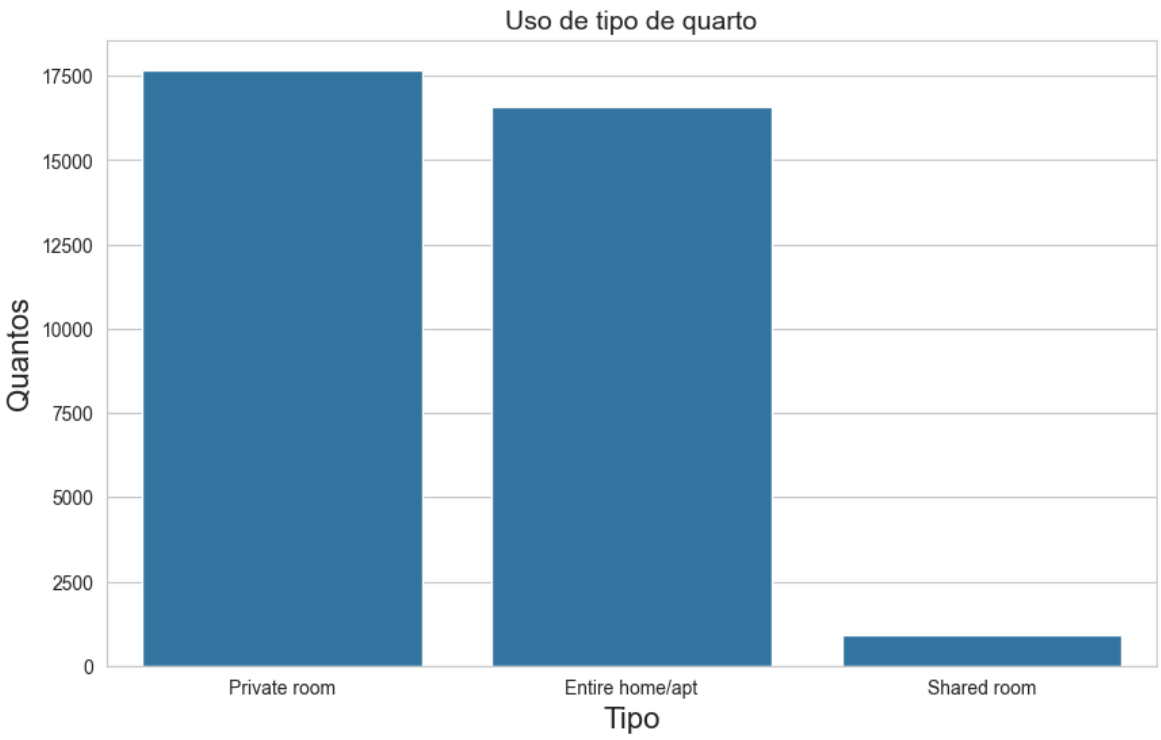
6.2) Análise por Tipo de Quarto (`room_type`)

Entender a distribuição dos tipos de quartos e como eles se relacionam com o preço é crucial para proprietários e hóspedes.

In [102…

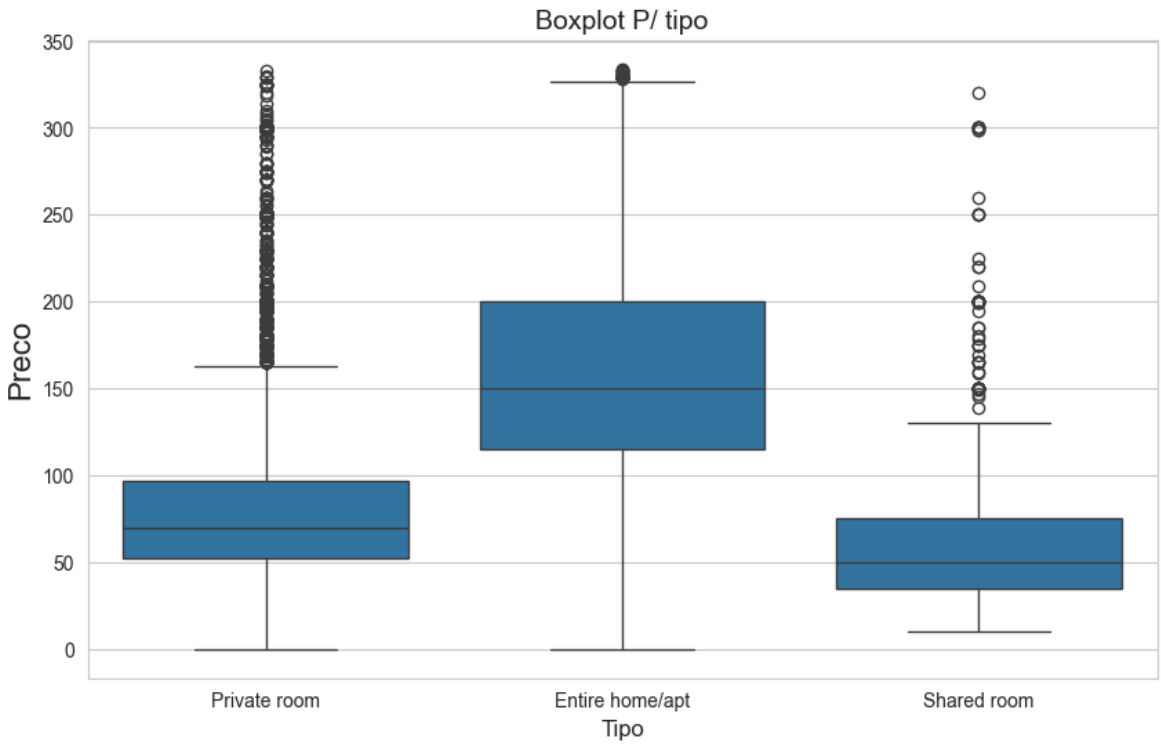
```
# Crie um gráfico de contagem dos tipos de quarto usando `sns.countplot`
# Crie um boxplot do preço por tipo de quarto usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(10, 6))
sns.countplot(x=df['room_type'])
plt.title("Uso de tipo de quarto")
plt.xlabel("Tipo",fontsize=16)
```

```
plt.ylabel("Quantos",fontsize=16)
plt.show()
sns.boxplot(data=df,x='room_type',y='price')
plt.title("Boxplot P/ tipo")
plt.xlabel("Tipo",fontsize=12)
plt.ylabel("Preco",fontsize=16)
```



Out[102…

```
Text(0, 0.5, 'Preco')
```

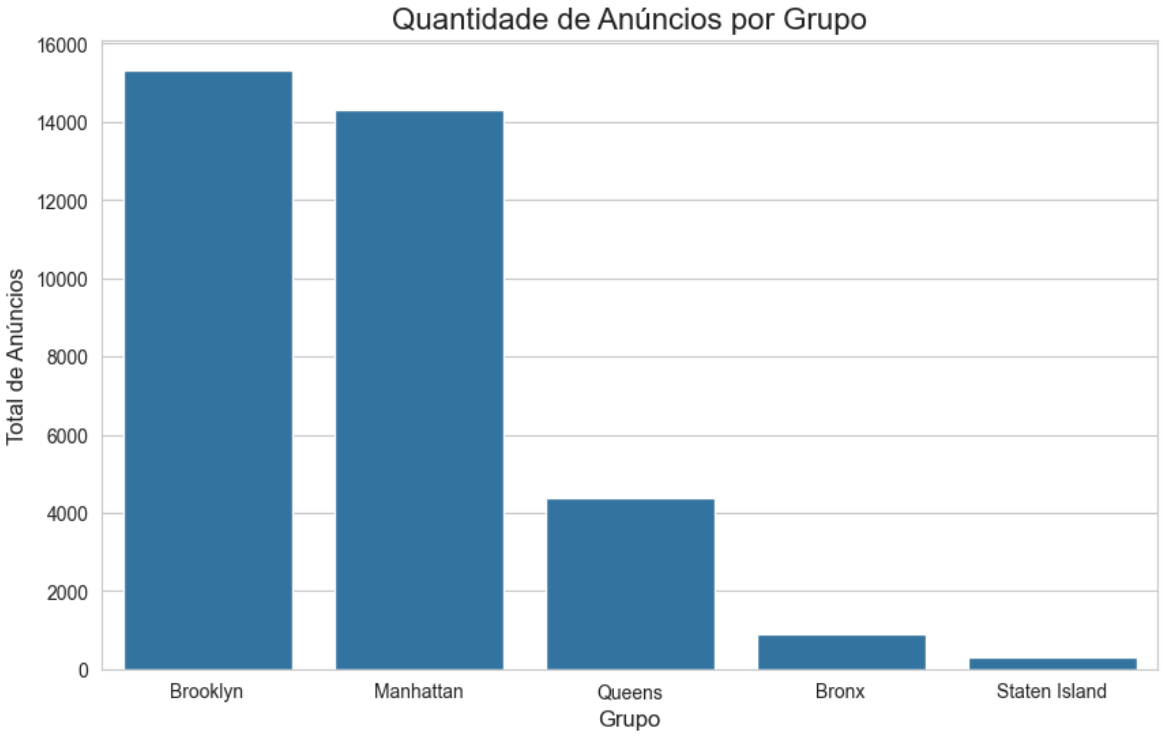


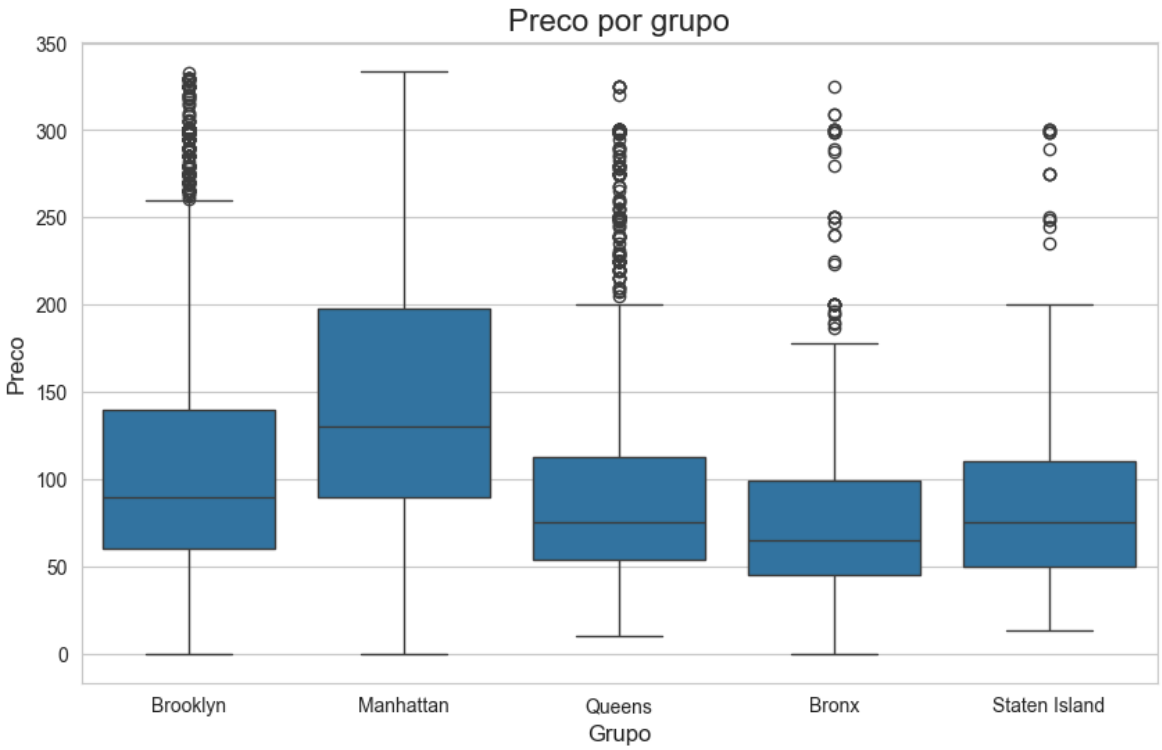
6.3) Análise por Grupo de Bairro (`neighbourhood_group`)

A localização é um dos fatores mais importantes no mercado imobiliário. Vamos analisar como os preços e a disponibilidade variam entre os diferentes grupos de bairros de NYC.

In [103…

```
# Crie um gráfico de contagem de anúncios por grupo de bairro usando `sns.countp
# Crie um boxplot do preço por grupo de bairro usando `sns.boxplot`
# Adicione títulos e rótulos aos gráficos
plt.figure(figsize=(10, 6))
sns.countplot(data=df, x='neighbourhood_group')
plt.title('Quantidade de Anúncios por Grupo', fontsize=16)
plt.xlabel('Grupo', fontsize=12)
plt.ylabel('Total de Anúncios', fontsize=12)
plt.show()
sns.boxplot(data=df,x='neighbourhood_group',y='price')
plt.title('Preco por grupo', fontsize=16)
plt.xlabel('Grupo',fontsize=12)
plt.ylabel('Preco',fontsize=12)
plt.show()
```



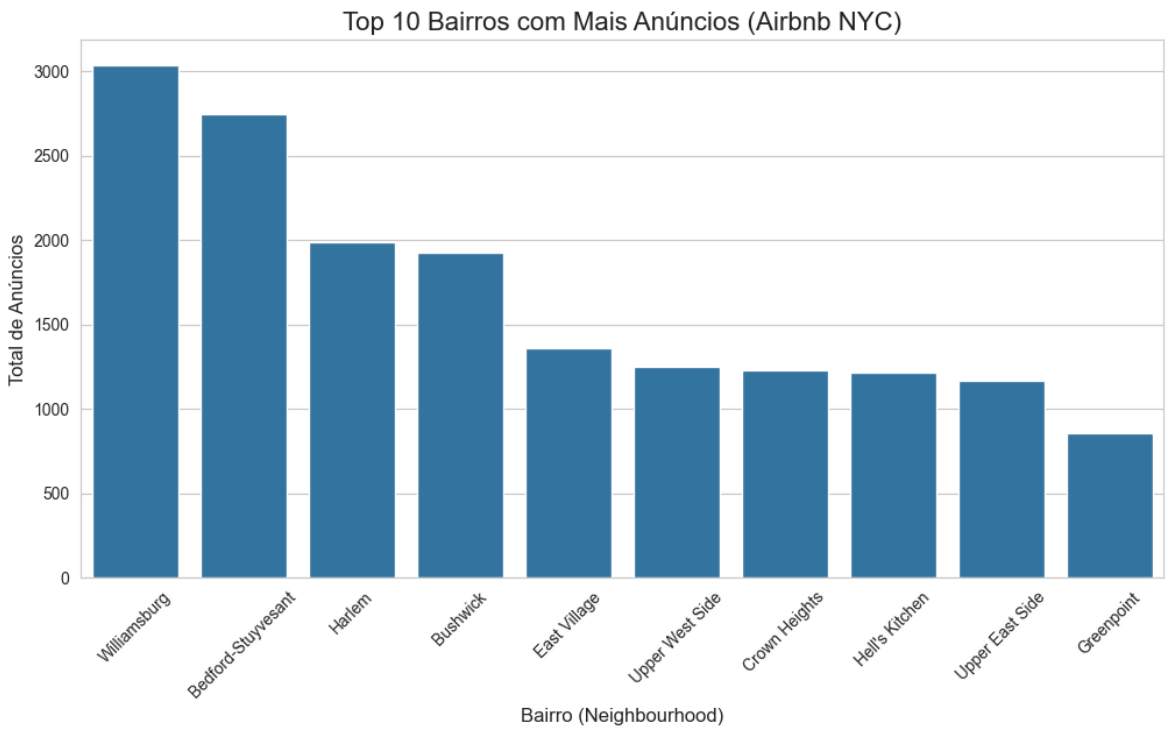


6.4) Top 10 Bairros com Mais Anúncios

Identificar os bairros com maior concentração de anúncios pode indicar áreas de alta demanda ou popularidade.

In [104…

```
# Calcule os 10 bairros com mais anúncios
# Crie um gráfico de barras para visualizar esses bairros usando `sns.barplot`
# Adicione títulos e rótulos aos gráficos, e rotacione os rótulos do eixo X se n
top_10_bairros = df['neighbourhood'].value_counts().head(10).reset_index()
plt.figure(figsize=(12, 6))
sns.barplot(data=top_10_bairros, x='neighbourhood',y='count')
plt.title('Top 10 Bairros com Mais Anúncios (Airbnb NYC)', fontsize=16)
plt.xlabel('Bairro (Neighbourhood)', fontsize=12)
plt.ylabel('Total de Anúncios', fontsize=12)
plt.xticks(rotation=45)
plt.show()
```



6.5) Análise Geoespacial dos Anúncios

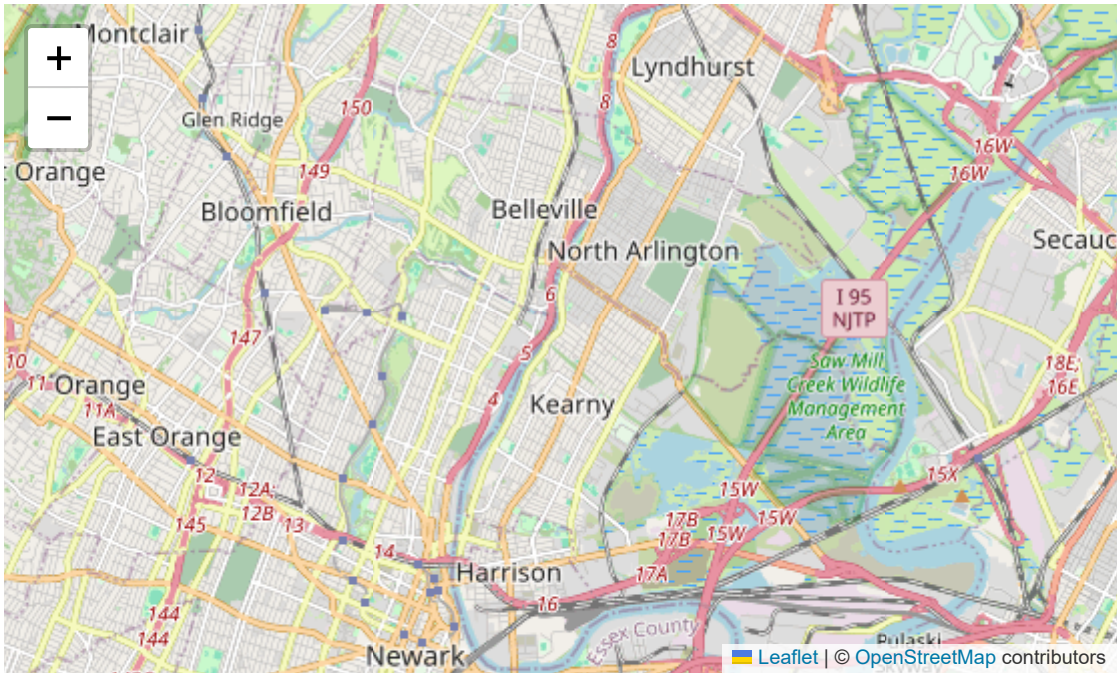
Visualizar os anúncios em um mapa nos permite identificar clusters geográficos e entender a distribuição espacial dos imóveis.

In [105…

```
# Crie um mapa base de Nova York usando `folium.Map`
# Adicione um `MarkerCluster` ao mapa
# Amostre uma parte do DataFrame (e.g., 5000 pontos) para melhor performance na
# Adicione marcadores ao mapa para cada ponto amostrado, incluindo informações n
# Salve o mapa em um arquivo HTML (e.g., "airbnb_nyc_map.html")
# (Opcional) Exiba o mapa diretamente no notebook se o ambiente suportar
from folium.plugins import MarkerCluster
mapa_nyc = folium.Map(location=[40.7128, -74.0060], zoom_start=12)
marker_cluster = MarkerCluster().add_to(mapa_nyc)
amostra = df.sample(n=5000, random_state=42)
for idx, row in amostra.iterrows():
    folium.Marker(
        location=[row['latitude'], row['longitude']],
        popup=f"Name: {row['name']}<br>Price: ${row['price']}<br>Room Type: {row
    ).add_to(marker_cluster)

mapa_nyc.save("airbnb_nyc_map.html")
mapa_nyc
```

Out[105…



7) Engenharia de Features

A engenharia de features envolve a criação de novas variáveis a partir das existentes para melhorar a capacidade preditiva dos modelos ou para obter novos insights. Aqui, criaremos algumas features simples.

In [92]:
```
# Calcule a data mais recente de `last_review`
# Crie a nova feature `host_experience_days` calculando a diferença em dias entr
# Exiba as estatísticas descritivas da nova feature
data_mais_recente=df['last_review'].max()
print(f"A data mais recente no dataset é: {data_mais_recente}")
print("-" * 40)
df['host_experience_days'] = (data_mais_recente - df['last_review']).dt.days
print(df['host_experience_days'].describe())
print("-" * 40)
```

```
A data mais recente no dataset é: 2019-07-08 00:00:00
----------------------------------------
count    35202.000000
mean       912.940032
std       1301.936154
min          0.000000
25%         22.000000
50%        190.000000
75%       1150.000000
max       3475.000000
Name: host_experience_days, dtype: float64
----------------------------------------
```

7.2) `price_per_night_per_review`

Esta feature pode indicar o custo-benefício percebido, relacionando o preço por noite com o número de avaliações. Um valor baixo pode indicar um bom negócio, enquanto um valor alto pode sugerir um preço elevado para poucos reviews.

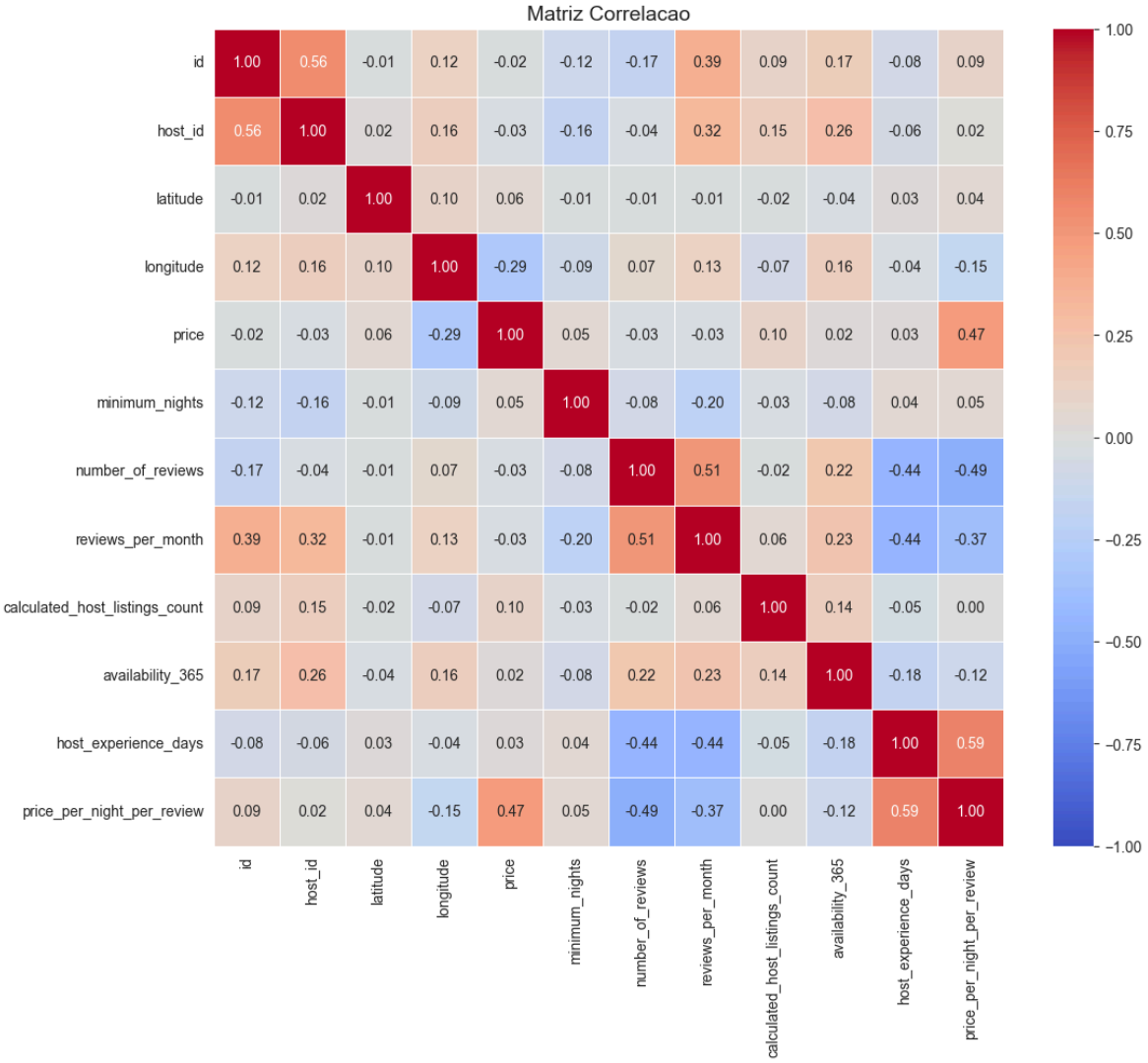
In [93]:
```
# Crie a nova feature `price_per_night_per_review`, dividindo `price` por `numbe
# Lembre-se de tratar a divisão por zero (e.g., se `number_of_reviews` for 0, us
# Exiba as estatísticas descritivas da nova feature
df['price_per_night_per_review']= np.where(
    df['number_of_reviews'] == 0,
    df['price'],
    df['price'] / df['number_of_reviews']
)
print(df['price_per_night_per_review'].describe())
```

```
count    35202.000000
mean        49.967902
std         63.791240
min          0.000000
25%          5.801613
50%         21.966667
75%         70.000000
max        333.000000
Name: price_per_night_per_review, dtype: float64
```

8) Análise de Correlação

A correlação mede a força e a direção da relação linear entre duas variáveis numéricas. Um mapa de calor (heatmap) da matriz de correlação é uma excelente ferramenta para visualizar essas relações.

In [95]:
```
# Selecione apenas as colunas numéricas do DataFrame
# Calcule a matriz de correlação
# Crie um heatmap da matriz de correlação usando `sns.heatmap`
# Adicione anotações e um mapa de cores adequado
# Adicione um título ao gráfico
colunas_numericas = df.select_dtypes(include=['number'])
matriz_correlacao = colunas_numericas.corr()
plt.figure(figsize=(12, 10))
sns.heatmap(matriz_correlacao, annot=True, cmap='coolwarm', fmt=".2f", vmax=1, v
plt.title('Matriz Correlacao')
plt.show()
```


Matriz Correlacao
id 1.00 0.56 -0.01 0.12 -0.02 -0.12 -0.17 0.39 0.09 0.17 -0.08 0.09
host_id 0.56 1.00 0.02 0.16 -0.03 -0.16 -0.04 0.32 0.15 0.26 -0.06 0.02
latitude -0.01 0.02 1.00 0.10 0.06 -0.01 -0.01 -0.01 -0.02 -0.04 0.03 0.04
longitude 0.12 0.16 0.10 1.00 -0.29 -0.09 0.07 0.13 -0.07 0.16 -0.04 -0.15
price -0.02 -0.03 0.06 -0.29 1.00 0.05 -0.03 -0.03 0.10 0.02 0.03 0.47
minimum_nights -0.12 -0.16 -0.01 -0.09 0.05 1.00 -0.08 -0.20 -0.03 -0.08 0.04 0.05
number_of_reviews -0.17 -0.04 -0.01 0.07 -0.03 -0.08 1.00 0.51 -0.02 0.22 -0.44 -0.49
reviews_per_month 0.39 0.32 -0.01 0.13 -0.03 -0.20 0.51 1.00 0.06 0.23 -0.44 -0.37
calculated_host_listings_count 0.09 0.15 -0.02 -0.07 0.10 -0.03 -0.02 0.06 1.00 0.14 -0.05 0.00
availability_365 0.17 0.26 -0.04 0.16 0.02 -0.08 0.22 0.23 0.14 1.00 -0.18 -0.12
host_experience_days -0.08 -0.06 0.03 -0.04 0.03 0.04 -0.44 -0.44 -0.05 -0.18 1.00 0.59
price_per_night_per_review 0.09 0.02 0.04 -0.15 0.47 0.05 -0.49 -0.37 0.00 -0.12 0.59 1.00
id
host_id
latitude
longitude
price
minimum_nights
number_of_reviews
reviews_per_month
calculated_host_listings_count
availability_365
host_experience_days
price_per_night_per_review
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00